

server2 provider budgets (S08)

Internal acquisition primitives for #2693. They do not enable a weather route, timer, key provisioning or deployment. See the admission sequence and specification.

1. Configure one trusted quota identity (S1/S2)

Construct `BudgetPolicy` with provider (`data_go_kr` or `visual_crossing`), a 64-character lowercase hexadecimal logical quota owner, stable provider-local `window_id`, explicit UTC epoch start/end milliseconds, positive `limit` and `block_units`. Default engineering recommendation is 256 units; the primitive takes explicit policy, with no silent defaults or daily reset. All processes using the same actual provider quota must use the same identity, limit, window and endpoint ownership mapping.

The library cannot prove that a provider credential belongs to that quota owner or that separate endpoints have separate ceilings. Candidate provider equality is only a cheap configuration check. S09 must verify the real key/quota/endpoint/provider-local-day mapping, trusted clock, no-delete/no-rollback policy and same-region IAM. These are prerequisites, not observations made by local tests. Separate server2 keys are still required before live coexistence. No actual keys or precise coordinates belong in policy IDs, logs or objects.

Use caller-managed `HttpS3Transport` credentials; refresh them coherently before expiry. Budget access is a separate `budgets/v2/` namespace, not the weather catalog. Required production missing-object GET=404 and conditional writes must be tested in S09 staging. Insufficient `ListBucket` permission can yield 403 instead of 404; the primitive denies funding, and this task neither widens IAM nor claims real signature verification.

2. Fund before HTTP (S1/S2)

`BudgetStore::reserve(maximum, absolute_tokio_deadline)` returns a private non-Clone reservation. Cold funding GETs authority, CAS PUTs the exclusive high-water range, then conditionally PUTs its immutable witness. Only a completely received HTTP 200 acknowledgment of both writes allows ownership. Header 200 followed by a truncated response is unknown.

The global ceiling lives at `budgets/v2/{provider}/{quota_id}/{window_id}/authority.json`. Witnesses live in `blocks/{start}-{end}.json` below that prefix. JSON contains only immutable policy and range identities/capacity. It contains no raw/normalized weather or request data. CAS enforces non-overlap under trusted conformant object semantics; it is not a multi-object transaction. At most eight CAS conflicts are attempted inside the original deadline. Their bounded exhaustion returns `Contention`; a real insufficient quota returns `Exhausted`.

Existing live owners reuse only their own unissued block remainder. Every request maximum and every candidate is debited before the first provider attempt. Units are never refunded if an attempt is unused, cancelled or fails. Replacement never reads old witnesses to reclaim leftovers. Oversized request maxima or insufficient final partial block capacity deny work.

A new block has three dependent S3 waves; warm owner funding has zero. Two cold key candidates are funded sequentially, so that request has six dependent reservation waves. Earlier 0.1–0.2 second illustrations are not measurements or this implementation's guarantee. Block waste and latency must be measured in S09. Up to each live block's capacity can be lost at replacement; multiple key blocks contribute separately. No local-peer number predicts real S3 latency.

3. Acquire and publish raw data (S4/S5)

`FundedExecutor::new(provider, per_attempt_units, transport, validator)` owns at most two actual HTTP attempts total. `Data.go.kr` uses one unit per call: the first candidate reserves two units and an optional rotation candidate reserves one more upfront. All three funded units burn even when rotation is unused. VC per-attempt costs of 49/25/1 give maxima of 98/50/2.

`execute(endpoint, candidates, deadline)` rejects cheap invalid targets before funding, including insecure non-loopback HTTP, embedded credentials and duplicate key/dataType query parameters. Duplicate rotation secret, shared `BudgetStore` or provider/quota owner is rejected before any funding or HTTP, including another window for the same owner. Errors omit secrets. `ProviderKey::new` expects the caller's decoded secret, which is encoded once by the query builder. Endpoint/kind/page parameters and key normalization compatibility remain provider-adaptor responsibilities. The HTTP client has no redirects, proxy discovery or implicit retries. `Data.go.kr` appends `dataType=JSON`, but XML error bodies are recognized.

HTTP 429/code 22 means quota; HTTP 401/403 or codes 20/30/31/32 means auth. Those stop same-key retry and may rotate to the pre-funded next key. Other 4xx stops. A complete recognized 2xx JSON/XML 03 envelope returns terminal `NoData` without

retry even if the data validator rejects it; 10/12 are terminal Rejected. HTTP 3xx/5xx +03 remains transient, auth/quota/other4xx has priority, and malformed/HTML 03 bodies are not no-data. Transport, 5xx, 3xx and empty or invalid bodies allow one funded subsequent attempt within the original deadline. JSON and XML code/status combinations are tested. The kind-specific Validator must validate semantic data; generic JSON syntax alone does not establish provider parity.

AcquisitionOutcome::Data(AcquiredBody) or NoData(AcquiredBody) holds exact raw bytes/status/ content type/attempt count in memory. NoData is not weather data: do not construct archive records or invented rows from it. Future provider ports choose legacy-compatible wire/empty semantics and any separately authorized persistence; the generic primitive stores none. For Data only: Caller S07 glue (not implemented here) must construct ordered raw records/declaration and complete S06 publication before new client success. Funding or raw HTTP is not group commit. The release example demonstrates this caller publication explicitly.

Reuse one executor per intended admission domain. It admits four executions, retaining the slot across funding, HTTP body download and CPU validation. Two CPU slots remain held by blocking closures even when the owner future is cancelled. Max HTTP body is 8 MiB; this bounds retained input per instance, not global RSS or JSON object expansion. Custom transports and validators are trusted internal code; validators should not block indefinitely. Creating unbounded executor instances defeats per-instance bounds. Future route-level shared admission and S07's cooperative owner shutdown are integration responsibilities.

4. Fail closed and recover (S3/S4)

Exhaustion/policy corruption/status rejection, time-window violation or clock rollback denies acquisition. Unknown CAS/witness, transport loss or timeout has no provider fallback. A committed unknown write remains charged; a never-committed/unrecorded write cannot be inferred after replacement. This limitation does not allow unreserved HTTP. Clock observation and its local monotonic fence are serialized. Stable policy and trusted clock after replacement remain operational prerequisites; monotonic deadlines do not repair wall-clock trust.

Dropping the owner acquisition future is the S08 cancellation mechanism. Cancelling a client waiter must not drop shared owner work; S07 distinguishes these lifetimes. Requests after expiry cannot revive a window. No automatic window creation, reservation refund, witness replay, deletion or rollback is provided.

Operational rollback disables new acquisition and preserves all budget objects. Investigate sanitized typed errors; do not delete the authority to unblock quota. Push delivery retry policy is unchanged. No shared-instance cache or SQLite is introduced.

5. Run the isolated release scenario (S5)

From the repository root, build and use only fresh owned loopback peers:

```
cargo build --manifest-path server2/Cargo.toml --locked --release --example provider_budgets_smoke
python3.11 server2/tests/budget/peer.py --binary server2/target/release/examples/provider_budgets_smoke
```

The test runner accepts only an owned release binary, binds ephemeral IPv4 loopback ports, uses obvious dummy credentials, and shuts down both peers. Fixtures are synthetic protocol examples, not recordings from live providers. It asserts quota XML rotation, complete S06 raw group publication/cold lookup, replacement exhaustion, committed unknown CAS and observed JSON/XML 03 exact RAM-only outcomes and 10/12 terminal rejection, each one HTTP and no raw publication. It observes eight provider requests, eighteen charged units and eight witnesses. S3 peer state is volatile. Tests validate protocol ordering and arithmetic, not IAM, signature crypto, actual provider quota, latency, public API/mobile parity or production durability after host loss.

The accompanying screenshot/PDF records one actual local execution; its capture date and source hashes are in the manifest. CI retains existing storage regressions plus budget tests. No route cutover or legacy retirement is authorized by this manual or PR.

Actual execution receipt

S08 ACTUAL LOCAL RELEASE EXECUTION

UTC 2026-10-07T22:03:42+00:00

\$ python3.11 server2/tests/budget/peer.py --binary

server2/target/release/examples/provider_budgets_smoke

cold: both candidates pre-funded; quota XML rotated key; 2 HTTP total

caller publication: raw body + complete S06 group; fresh S3 lookup matches bytes

replacement: old unused/issued funds never reclaimed; exhausted request sends 0 HTTP

unknown committed CAS: no HTTP; replacement reserves disjoint next range

terminal JSON/XML03: one HTTP each, exact bytes RAM-only; 10/12 rejected without retry

PASS synthetic local release smoke; no AWS signature/IAM/latency or API parity claim

observed peer counters: 8 provider requests, 18 charged units, 8 witnesses; PASS